Learning Objectives

This lab lets you practice `for` and `while` loops (including branching in the loops)

Estimated Size

3 program files: `is_prime.py, for_count.py, zigzag.py`
Each should be under 10 lines of code.

Setup

In your Python files, you must include author information. On the first line, add a **`# Author`** comment line, where you mention your full name. Similarly, include your email and Spire ID on the **`# Email`** and **`# Spire ID`** comment lines, respectively. Remember, these comments must appear **at the beginning of every Python file** that you submit to Gradescope. For example:

```
# Author   : John Snow
# Email    : jsnow@umass.edu
# Spire ID : 12345678
```

`while` loop basics

In lecture 9, you learned Python's `while` loop. A typical `while` loop goes like this: before the loop, you usually have some instructions to create and initialize variables to be used and updated in the loop; then you write the keyword `while` followed by a boolean expression that is the loop condition, and a colon after that; next is an indented block of code which is the loop body. The loop body runs repeatedly while the condition is `True`, and terminates when the loop condition becomes `False`. For example, a loop that prints out the first `n` integers, one on each line, can go like this:

```
n = 20
i = 1          # initialize loop control variable i
while i<=n:    # loop condition
  print(i)     # print i
  i += 1       # increment loop control variable i
```

Here, the variable `i` is often referred to as the 'loop control variable' because it controls how many iterations the loop runs. It starts with a value of 1, and with each iteration, it increments by 1. The loop runs until `i` is no longer smaller than or equal to `n`, so in total this loop will run `n` iterations. In each iteration, the loop body prints out the value of `i`. Over `n` iterations, the value of `i` increases from 1 to `n`, so this program will print out the first `n` integers, one on each line.

1. Implement `is_prime.py` (5 points)

The first exercise is to write a **function** to check if a given integer is prime or not. A prime number is defined as a positive integer larger than 1, that is only divisible by 1 and itself, and not divisible by any other integer. For example, 5 is a prime number because it's only divisible by 1 and 5 (itself); 12 is NOT a prime number because it's divisible by 3 and 4 in addition to 1 and itself.

Given an integer `n`, one way to check whether it's prime is to check if it's divisible by any integer from 2 to $\sqrt{n}$ **inclusive on both ends** (you may use `n**0.5` to compute $\sqrt{n}$ and cast it to an integer). If true, it is NOT a prime; otherwise, it's a prime. For example, to check if 17 is prime, it's sufficient to check if 17 is divisible by any integer from 2 to 4 ($\sqrt{17} \approx 4.123$, cast to 4). Since 17 is not divisible by any of 2, 3, 4, it is a prime.

Specifically, you should:

- Create a file named `is_prime.py`, and add your info as comments at the top
- Write a function, called `is_prime`, that takes an integer `n` as a parameter, and returns a boolean value representing whether it's prime.
- Inside the function, you should write a `while` loop that iterates from `2` to `int(`$\sqrt{n}$`)` inclusive on both ends and check if `n` is divisible by any of them (think about how to check if one integer is divisible by another). If so, you can directly `return False` (i.e., not a prime) in the loop. A `return` in a loop causes the loop to terminate immediately and causes the function to exit as well.
- After the loop, think about what one instruction you should put there. *Hint*: this would be where the loop is done, and it didn't find any divisor of integer `n`, which indicates `n` is a prime.

Below are examples of running a correct implementation of this function:

```
print(is_prime(15))      ⇒ False
print(is_prime(17))      ⇒ True
print(is_prime(25))      ⇒ False
print(is_prime(26))      ⇒ False
print(is_prime(97))      ⇒ True
```

Requirement: your program must contain **a function called** `is_prime`. It must take an integer as a parameter and must **return a boolean value**. Do **NOT** print anything inside this function. Do **NOT** use the input function before the function.

`for` loop basics

In lecture 10, you learned Python's `for` loop. Compared to a `while` loop, a `for` loop is far more convenient for iterating over sequence data types (e.g., lists, tuples, characters in strings) and iterating over a sequence of numbers. It can also help prevent unintentional infinite loops (which is possible with a `while` loop in case you forget to update the loop control variable).

To iterate over elements in a sequence type object, you write the keyword `for`, followed by a variable name of your choice, then the keyword `in`, followed by the name of the sequence type object. Next is an indented block of code which is the loop body. In the loop body, you can use the variable name of your choice to get the value of the element in the current iteration. For example, a loop that prints out every element in a list, one on each line, can go like this:

```
simpsons = ['Homer', 'Marge', 'Lisa', 'Bart', 'Maggie']  # a list of
strings
for name in simpsons:
  print(name)
```

To iterate over a sequence of numbers, instead of creating a list of numbers manually, you can use the `range()` function: `range(a, b)` generates the integer sequence `[a, b)`, `a` inclusive, `b` exclusive, or in other words, the sequence `a, a+1, a+2, ... b-1`. For example, a loop that prints out all integers from 100 to 200 (200 included), one on each line, can go like this:

```
for number in range(100, 201):   # note: 201 as you want 200 to be included
  print(number)
```

To iterate over a sequence of numbers in reverse order, you can use slicing with `range()` function. For example, a loop that prints out all integers from 200 to 100 (200 included), one on each line, can go like this:

```
for number in range(100, 201)[::-1]:   # note: 201 as you want 200 to be
included
  print(number)
```

Now, with loops and branching statements together, you can solve more interesting and complex problems. For example, to count how many vowels there are in a string, you can do:

```
some_string = 'How many vowels are there in this string?'
count = 0                          # initialize the count variable
for chr in some_string:            # loop over characters in the string
  if chr in 'aAeEiIoOuU':          # check if the character is a vowel
    count += 1                     # if True, increment count
print(f'There are {count} vowels in the string')
```

As another example, imagine you have the `is_prime` function (from lab 5) ready to check if a given integer is prime or not, you can now answer more complex questions like how many prime numbers there are between `a` and `b` (b included):

```
count = 0                          # initialize the count variable
for number in range(a, b+1):       # note: b+1 as you want b to be included
  if is_prime(number):             # call is_prime to check if n is prime
    count += 1                     # if True, increment count
```

```
print(f'There are {count} prime numbers between [{a}, {b}]')
```

2. Implement function `count_strings` (5 points)

Create a new file in VSCode and save/name it `for_count.py`. Your first exercise is to write a function called `count_strings` which takes a **list of strings** and **an integer** `n`, and returns how many strings in the list **contain at least** `n` **characters**. Specifically, your function should

- Take a list of strings and an integer `n` as parameters
- Use a `for` loop to iterate over the strings and count how many strings have `n` or more characters (you may use the `len()` function to calculate the number of characters in a string)
- Return the count
- Do **NOT** ask the user for any input. Do **NOT** print anything in your function.

When you are done, write some test cases yourself to check if the return value of your function is as expected. For example:

```
count_strings(['', 'a', 'aa', 'aaa'], 0)   # should return 4
count_strings(['', 'a', 'aa', 'aaa'], 2)   # should return 2
count_strings(['', 'a', 'aa', 'aaa'], 4)   # should return 0
```

Gradually, you will learn to write tests that can check the so-called 'edge cases' too -- cases where the input parameters are at extreme values. For example, does your function return the correct value if an empty list is given to it? What about when `n` is 0, or a very large number? These edge cases can often help you identify where your code may contain a bug or where it needs to be fixed.

3. Implement `zigzag.py` (5 points)

Write a function, called `is_zigzag`, inside the file **zigzag.py**, which should:

- Take a list of integers as a parameter
- Return True if the list follows a **zigzag pattern**, and False otherwise
- A list with fewer than 3 elements should return True (since there's no middle element to violate the rule)

Definition:
A list is **zigzag** if every element (except the first and last) is either:

- Strictly greater than both its neighbors, or
- Strictly smaller than both its neighbors.

Important notes:

- The return type must be **boolean**, not string.
- You must use a **for loop** to check all middle elements.
- Do **not** modify the input list.

For example:

```
print(is_zigzag([1, 3, 2, 4, 3]))    # True
print(is_zigzag([1, 4, 2, 5, 3]))    # True
print(is_zigzag([1, 2, 3, 4]))       # False
print(is_zigzag([10]))               # True
print(is_zigzag([1, 3, 2, 4, 5]))    # False
```

4. Submit your code to Gradescope

Log in to Gradescope, select **Lab 05: Loops (Code)**, and submit **all files**. Wait until the autograder finishes running and returns the result to you. You can resubmit as many times as you like before the deadline.

5. Submit the Lab Questionnaire on Gradescope

Remember to submit the Lab 5 questionnaire too.

If you see a question you do not know how to answer, ask TAs/UCAs in your lab, use Piazza, or go to one of our office hours.

(Common Paragraph) Academic Honesty

All work that is completed in this assignment is your own (if you work individually) or your own group's (if you work in a group). You may talk to other students about the problems, and brainstorm about solutions. However, you may NOT share code in any way, except with your group members. What you submit **must be your own or your own group's work**.

You may not use any code that is posted on the internet. If you are not sure, it is in your best interest to contact the course staff. We will be using software that will compare your code to other students in the course as well as online resources. It is very easy for us to detect similar submissions, which will result in a failure for the exercise or possibly a failure for the course. Please, do not do this. It is important to be academically honest and submit your work only. Please review the UMass Academic Honesty Policy and Procedures so you are aware of what this means.

Copying solutions, whether partial or whole, obtained from other students or elsewhere, is academic dishonesty. Do not share your code with your classmates, and do not use your classmates' code. If you are confused about what constitutes academic dishonesty, you should re-read the course policies. We assume you have read the course policies in detail, and by submitting this project, you have provided your virtual signature in agreement with these policies.